

LLM-Based Rubric Grading for Programming Assignments: An Empirical Study

Alfonso Piscitelli^a, Mattia De Rosa^b, Vittorio Fuccella^c and Gennaro Costagliola^d

Department of Informatics, University of Salerno, Fisciano (SA), Italy

Keywords: Large Language Models, Automatic Code Grading, Rubric-Based Evaluation.

Abstract: Automatic grading systems support the evaluation of student submissions by reducing teachers' workload while providing rapid and consistent feedback. Recent advances in Large Language Models (LLMs) have expanded the potential of automated assessment in programming education, enabling evaluation beyond functional correctness to include aspects of code quality. However, grading can be affected by subjective interpretations of evaluation criteria, and structured rubrics can help guide the process and reduce variability. This study investigates the use of LLMs for rubric-based automatic grading of programming assignments. We evaluate thirty student submissions previously graded by a teacher using a rubric-based grid. Three LLMs (gpt-4o, gpt-5-mini, and qwen-coder:32b) were used to assess the same dataset, and the generated grades were compared with the teacher's ones. Additionally, we analyze agreement across individual rubric dimensions to identify aspects that are more challenging for LLM-based evaluation. Results show that rubrics help LLMs produce grades closer to human evaluations. However, indicators such as Specification Adherence and Autonomy show larger discrepancies, suggesting difficulties in interpreting assignment instructions.

1 INTRODUCTION

Automatic grading of student submissions is a research area that investigates the use of automated systems to evaluate student work and provide feedback and grades. The use of automatic grading systems has the potential to reduce both the workload of teachers and the biases that may arise in human grading, while providing more consistent and objective evaluations. Moreover, providing rapid feedback to students can help them identify their mistakes autonomously, improving their learning process and allowing them to correct errors before receiving the final grade.

In recent years, the emergence of Large Language Models (LLMs) has opened new possibilities for automatic grading, allowing the evaluation of complex submissions by leveraging the natural language understanding capabilities of these models. In the context of computer science and programming education, LLMs have been used to evaluate programming as-

signments, providing an analysis that goes beyond the simple correctness of the output, generally measured through test cases, to include other aspects of code quality, allowing to provide a more comprehensive evaluation of student submissions. However, the grading process is challenging also for human teachers, who may have different interpretations of the grading criteria, and may be influenced by various biases, such as the halo effect, which can lead to inconsistent grading Messer et al. (2025).

In this context, the use of a structured rubric can help to guide both human and LLM grading processes, providing clear criteria and descriptors for each level of evaluation. Despite several studies have investigated the use of LLMs for automatic grading with rubrics, the use of structured rubrics with multiple weighted indicators and the analysis of LLM reliability at the indicator level remains underexplored.

In this study, we investigate the use of LLMs for automatic grading of programming assignments using a rubric-based approach, to evaluate thirty programming assignments submitted by students in programming courses, and previously evaluated by a teacher using a rubric-based evaluation grid. We tested three different LLMs, *gpt-4o*, *gpt-5-mini* and

^a <https://orcid.org/0000-0002-3567-1504>

^b <https://orcid.org/0000-0002-6922-5529>

^c <https://orcid.org/0000-0003-3244-3650>

^d <https://orcid.org/0000-0003-3816-7765>

qwen-coder:32b, to evaluate the same dataset, and we compared the grades generated by the LLMs with the grades assigned by the teacher, to evaluate the agreement between human and LLM grading. To evaluate the stability of the grading process, we ran each LLM five times with the same configuration and dataset, and we analysed the agreement across repeated runs, to evaluate the consistency of the grading process. Finally, we also analysed the agreement between LLM-generated and teacher-assigned grades across different rubric dimensions to evaluate if there are specific dimensions that are more difficult for LLMs to evaluate.

Specifically, this study aims to answer the following research questions:

- **RQ1:** What is the difference between grades generated by an LLM and those assigned by educators when evaluating programming assignments using a grading rubric?
- **RQ2:** How consistent are LLM-based evaluations across repeated executions under identical conditions?
- **RQ3:** Does the agreement between human and LLM grading vary across different rubric indicators?

The remainder of this paper is structured as follows: Section 2 presents the related work in the field of automatic grading and rubric-based evaluation. Section 3 describes the research methodology, including the dataset, the rubric used for evaluation, the LLMs configuration, and the evaluation metrics. Section 4 presents the results of the study, addressing each research question. Finally, Section 5 discusses the implications of the findings and outlines future research directions.

2 RELATED WORK

The grading of programming assignments is a central component of computer science education. However, manual grading requires a significant time investment on the part of teachers and can be subject to inconsistency among graders. In recent decades, numerous solutions have been proposed to automate or support the grading process, initially through systems based on automated testing and more recently through machine learning techniques and Large Language Models (LLM). In this section, we discuss the main research areas relevant to our work.

2.1 Reliability of Human Grading

Several studies have shown that human grading of student work can vary significantly. A meta-analysis conducted by Falchikov and Goldfinch demonstrated that agreement between peer and teacher assessments can vary considerably, particularly when the assessment process involves multiple criteria or complex judgments Falchikov and Goldfinch (2000).

Even when structured rubrics are used, the consistency of evaluations is not guaranteed. Bloxham et al. have shown that the interpretation of evaluation criteria can vary among evaluators, since evaluation judgments involve interpretive and tacit processes that make a certain degree of variability inevitable Bloxham et al. (2016).

In the specific context of computer science education, Messer et al. analyzed the consistency of grades given to introductory programming assignments, finding low levels of agreement among graders even when a shared rubric was used Messer et al. (2025). Studies of this kind highlight how the evaluation of programming assignments can exhibit substantial variability even when shared rubrics are used. This variability motivates the exploration of automated approaches that may support more consistent grading processes.

2.2 Automatic Grading of Programming Assignments

The automatic grading of programming assignments has been a subject of research for several decades. Early systems relied primarily on running automated tests to verify the functional correctness of students' programs. Douce et al. (2005) provides an influential review of test-based automatic grading systems, highlighting their benefits in terms of scalability and rapid feedback Douce et al. (2005). Some educational tools have also introduced environments that allow students to test their programs and receive immediate automated feedback, thereby encouraging the development of software testing skills within programming courses Edwards (2003).

Subsequently, Ihantola et al. (2010) analyzed the development of automated assessment systems in the context of programming education, discussing the various features offered by these tools and the main technical and pedagogical challenges Ihantola et al. (2010).

In addition to simply verifying the correctness of a program, several studies have explored techniques for generating more detailed feedback on students' errors. For example, Singh et al. proposed an approach based on program synthesis to automatically

generate minimal corrections for incorrect programs and provide feedback on the errors in students' solutions Singh et al. (2013).

Other studies have used machine learning techniques to model patterns in students' solutions and propagate feedback across similar problems, as in the work by Piech et al. (2015).

A more recent systematic review conducted by Keuning et al. (2018) highlights that many automated evaluation solutions continue to focus primarily on the functional correctness of code, whereas the evaluation of qualitative aspects of code, such as readability, structure, and documentation, is significantly more difficult to automate than simply verifying functional correctness Keuning et al. (2018).

2.3 Large Language Models for Automated Grading

Recent advances in LLMs, including models trained on large amounts of source code, have opened up new possibilities for automated grading. Thanks to their ability to understand both natural language and source code, LLMs can potentially assess complex qualitative dimensions. Early discussions on the potential of LLMs in education, such as Zhai (2023), have discussed the possible use of ChatGPT to support not only grading but the entire process of educational assessment Zhai (2023). Today, LLMs are being studied to evaluate qualitative aspects of code that are difficult to analyze using traditional algorithmic approaches, such as readability, structure, and documentation quality.

Several recent studies have explored the use of LLMs for the automatic grading of programming assignments, proposing both evaluation frameworks and models trained or adapted for code grading Mohamed et al. (2025); Yousef et al. (2025). Pathak et al. (2025) propose an approach based on exercise-specific rubrics, showing that embedding question-specific rubrics in the prompt significantly improves the quality of model-generated evaluations and their alignment with human assessments compared to question-agnostic rubric designs Pathak et al. (2025). Similarly, Mohamed et al. (2025) conduct an empirical analysis of the performance of various LLMs in grading programming assignments, highlighting that state-of-the-art models can achieve high levels of agreement with teacher grades across several evaluation tasks Mohamed et al. (2025). Other studies have proposed specific systems for automatic grading based on LLMs. For example, Cisneros-González et al. (2025) present JorGPT, a system that allows teachers to evaluate programming assignments

using different LLMs within a configurable rubric-based interface Cisneros-González et al. (2025). Similar approaches have also been explored by Tseng et al. (2024), who propose an automatic grading framework that combines Chain-of-Thought prompting with LLM ensembles to improve the consistency and reliability of grading and feedback Tseng et al. (2024). Akyash et al. (2025) proposed Step-Grade, evaluating 30 Python assignments on dimensions such as functionality and code quality Akyash et al. (2025).

The use of LLMs for grading has also been studied in broader educational contexts. For example, Xie et al. (2026) propose a multi-agent framework, Grade-Like-a-Human, designed to replicate typical human grading practices such as consistency checking, although it focuses on the evaluation of open-ended textual responses (short-answer grading) Xie et al. (2026). Similarly, Schneider et al. analyze the potential of LLMs for the autograding of open-ended responses, highlighting the need for human supervision Schneider et al. (2024). In parallel, Grandel et al. explored the scalability of GPT-4 in assessing functional programming assignments, demonstrating how well-defined rubrics and careful prompt engineering can improve the objectivity and efficiency of grading Grandel et al. (2024). In the domain of programming, Jukiewicz conducted a systematic comparison highlighting how model architecture significantly influences the stability and consistency of the evaluations produced Jukiewicz (2025). Beyond educational assessment, several studies have explored the use of LLMs for program analysis tasks, demonstrating their ability to reason about source code structure, detect potential errors, and support software analysis workflows. For example, Mahbub et al. use GPT-4 to detect ambiguities and inconsistencies in software requirements Mahbub et al. (2024), while Chapman et al. propose integrating static analysis with LLM prompting to improve error detection in code Chapman et al. (2024).

Overall, recent research shows promising but still mixed results, with significant differences across models, prompting strategies, and application contexts.

In particular, Piscitelli et al. compared the performance of various models (GPT-4o, Claude 3.5 Sonnet, Llama 3) in grading student code, highlighting how the choice of prompting strategy is critical to the accuracy of the final grade Piscitelli et al. (2025). However, the use of LLMs is not limited to assessment alone: previous studies have analyzed how the use of chatbots influences students' workflows, finding a reduction in completion times but also risks as-

sociated with excessive reliance on the model for generating complete solutions Piscitelli et al. (2024).

2.4 Research Gap

Despite growing interest in the use of LLMs for grading, several critical issues remain. Most of the existing literature focuses on high-level languages such as Python or Java, or on tasks involving the evaluation of textual responses Akyash et al. (2025); Grandel et al. (2024); Xie et al. (2026). Empirical data regarding lower-level languages, such as C, remains relatively limited.

Most existing studies focus on assessing the overall agreement between scores generated by LLMs and those assigned by humans, often relying on a single aggregate score or a limited set of evaluation criteria. Less attention has been paid to analyzing how agreement with human evaluators and the stability of LLM evaluations vary across the individual dimensions of a structured grading rubric. In particular, it remains unclear which aspects of code quality, such as functional correctness, maintainability, readability, or documentation, can be reliably evaluated by LLMs.

3 METHODOLOGY

This section outlines the research methodology, the proposed software architecture, the experimental dataset, and the specific metrics evaluated during the software's execution.

3.1 Dataset

The dataset consists of programming assignments submitted by students in an introductory programming course. Each submission was manually evaluated by a teacher using a structured rubric.

The dataset includes a total of 30 programming submissions. Each submission includes the source code, the compilation output, the expected output and the output generated by the code execution. The dataset also includes the corresponding evaluation produced by the assistant teacher, who used the same grid of the LLM. The manual evaluations serve as the ground truth against which the LLM-based assessments are compared.

3.2 Rubric and Evaluation Scheme

Student submissions were assessed using a structured rubric adapted from the framework proposed by

Messer et al. Messer et al. (2025). The rubric comprises four quality indicators: correctness, code readability, maintainability, and documentation. Each indicator is assessed on a six-level ordinal scale ranging from 0 (non-compliance) to 5 (full compliance). To ensure consistent interpretation by both human evaluators and the LLM, each level is accompanied by explicit descriptors that define the expected characteristics of submissions at that level. The complete evaluation rubric is available online ¹.

3.3 Experimental Framework

The automatic grading process is implemented as a pipeline in which each student submission is evaluated by an LLM. For each case, the system generates a JSON input containing the C source code, program output, static analysis errors, expected output, and a flag indicating Makefile correctness.

The LLM is guided by a system prompt that defines its role, specifies a 0–5 grading scale for each indicator, and prescribes both the evaluation procedure and output format. The model returns a structured JSON response including scores and brief justifications, enabling automated comparison with instructor assessments.

The complete system prompt used in this study is provided in the Appendix. The prompt describes the task, the input format, the evaluation grid, the expected output format with an example.

In this experiment, we used the following Large Language Models (LLMs) for automatic grading:

- **gpt-4o**, with temperature set to 0.5 (to reduce the non-determinism effect) and context window of 8192 tokens.
- **gpt-5-mini**, with temperature set to 1 (the only available configuration) and context window of 8192 tokens.
- **qwen-coder:32b**, with temperature set to 0.5 (to reduce the non-determinism effect) and context window of 8192 tokens.

The three models were selected to represent different segments of the LLM landscape: a well-established commercial model widely adopted in prior research (*gpt-4o*), a cost-effective variant of the current state-of-the-art (*gpt-5-mini*), and an open-source code-specialized model deployable locally (*qwen-coder:32b*). This selection addresses practical concerns in academic settings, including API costs and hardware sustainability.

¹<https://alfy91.github.io/csedu-2026-appendices/>

To analyse the stability of the grading process, we ran the LLM-based evaluation process five times, using the same configuration and dataset, to observe the variability of the generated grades across runs.

3.4 Evaluation Metrics

The following metrics are used to quantify agreement between human and LLM-based grading, as well as the internal consistency of the model:

- **Exact Agreement, (EA):** the percentage of cases in which the LLM rating exactly matches the human rating.
- **Partial Agreement, (PA):** the percentage of cases in which the LLM rating matches at least four times the human rating.
- **Majority Agreement (MA):** the percentage of cases in which the majority of the five LLM runs matches the human rating.
- **Tolerance Agreement (TA):** the percentage of cases in which the LLM rating differs from the human rating by at most one level on the ordinal scale.
- **Run Agreement (RA):** the percentage of submissions for which all five LLM runs produce the same rating, providing an estimate of the stability of the LLM grading process.
- **Cohen's kappa:** a statistical measure of inter-rater agreement that accounts for agreement occurring by chance, used to evaluate both the agreement between LLM and human ratings and the consistency across LLM runs.

4 RESULTS

This section presents the findings relative to the three research questions guiding this study. The complete experimental procedure required approximately ten hours to execute; the majority of this duration was attributable to the *qwen-coder:32b* model, which was deployed locally on the researchers' hardware. Conversely, the remaining models evaluated in this study required approximately one hour each to process the dataset described in the preceding section

4.1 Grade Difference with Teacher

In this section, we address the research question RQ1: *What is the difference between grades generated by*

an LLM and those assigned by educators when evaluating programming assignments using a grading rubric?

Table 1 summarizes the difference between the grades generated by three different LLMs and the grades assigned by the teacher for each of the nine indicators defined in the rubric. The table reports the median grade assigned by each LLM for each indicator, the difference between the LLM median and the teacher grade, and the standard deviation of the LLM grades across the five runs. The results show that *gpt-4o* achieved the lowest general difference, with a total difference of +0.16 across all indicators, followed by *qwen-coder:32b* with a total difference of -0.64 and *gpt-5-mini* with a total difference of -0.50. The indicators with the highest positive difference were *Specification Adherence* for *gpt-4o* (+0.76) and *gpt-5-mini* (+0.51), while the indicator with the highest negative difference was *Robustness* for *qwen-coder:32b* (-0.73). However, *qwen-coder:32b* demonstrated superior performance in specific indicators, achieving the lowest difference in *Specification Adherence* (+0.01), *Naming*, and *Formatting*.

4.2 Agreement Intra-LLM

In this section, we address the research question RQ2: *How consistent are LLM-based evaluations across repeated executions under identical conditions?* Table 2 describes the agreement level of each LLM across five independent executions, showing both the total agreement (cases where the LLM gave the same level in all 5 executions) and the partial agreement (cases where the LLM gave the same level in at least 4 out of the 5 executions). The results showed that *gpt-4o* achieved total agreement in 60.4% of cases, followed by 54.5% for *gpt-5-mini* and 51.5% for *qwen-coder:32b*. Partial agreement was higher for all three models, ranging from a minimum of 77.4% for *qwen-coder:32b* to a maximum of 82.2% for *gpt-4o*. An analysis of the table shows that *gpt-4o* achieved the highest agreement in evaluating *functional completeness*, *design autonomy*, and *maintainability*, while *gpt-5-mini* performed best in evaluating *design autonomy*, *maintainability*, *modularity*, and *naming*. Finally, *qwen-coder:32b* achieved the highest agreement in evaluating *design autonomy* and *functional completeness*. The indicator that received the lowest total agreement was *Specification Adherence*, suggesting that LLMs have difficulty with the assignment instructions.

Figure 1 presents a scatter plot comparing each LLM's agreement with the teacher (y-axis) against its stability across the five runs (x-axis), measured using Cohen's κ . The results show that *qwen-coder:32b*

Table 1: Comparison of LLM grades with teacher grades across rubric indicators. Med. = average of the median grades across all submissions, Diff = average difference between the median LLM grade and the teacher grade per submission.

ID	Indicator	GPT-4o		GPT-5-mini		Qwen-Coder	
		Med.	Diff. (SD)	Med.	Diff. (SD)	Med.	Diff. (SD)
1	Functional Completeness	2.57	+0.22 (1.30)	2.47	+0.08 (1.25)	2.53	+0.23 (1.18)
2	Robustness	2.27	-0.45 (0.93)	2.07	-0.65 (0.80)	2.03	-0.73 (0.79)
3	Design Autonomy	2.53	+0.25 (0.95)	2.83	+0.51 (1.03)	2.00	-0.35 (1.13)
4	Specification Adherence	4.30	+0.76 (0.82)	2.63	-0.93 (0.55)	3.50	+0.01 (1.06)
5	Modularity	3.03	-0.05 (0.54)	2.93	-0.13 (0.58)	2.63	-0.43 (0.59)
6	Maintainability	2.73	-0.45 (0.55)	2.73	-0.44 (0.56)	2.17	-0.97 (0.61)
7	Naming	3.27	+0.35 (0.53)	3.23	+0.37 (0.57)	2.92	+0.02 (0.57)
8	Formatting	3.33	-0.27 (0.49)	3.67	-0.03 (0.55)	3.78	+0.08 (0.66)
9	Documentation	2.87	+0.11 (1.29)	2.10	-0.62 (1.16)	2.13	-0.62 (1.50)
TOTAL (max 10)		5.88	+0.16 (0.95)	5.22	-0.50 (0.83)	5.08	-0.64 (1.07)

Table 2: Stability of LLM grading across five independent runs. The table shows the percentage of cases where the LLM assigned the same level across all five runs (Exact Agreement, EA%) and the percentage of cases where it assigned the same level in at least four out of five runs (Partial Agreement, PA%).

ID	Indicator	GPT-4o		GPT-5-mini		Qwen-Coder	
		EA (%)	PA (%)	EA (%)	PA (%)	EA (%)	PA (%)
1	Functional Completeness	70.0	83.3	46.7	80.0	66.7	90.0
2	Robustness	63.3	83.3	63.3	83.3	53.3	76.7
3	Design Autonomy	76.7	90.0	63.3	83.3	83.3	93.3
4	Specification Adherence	36.7	66.7	50.0	70.0	36.7	60.0
5	Modularity	63.3	86.7	66.7	86.7	36.7	70.0
6	Maintainability	66.7	90.0	63.3	80.0	36.7	76.7
7	Naming	60.0	83.3	43.3	73.3	50.0	80.0
8	Formatting	50.0	70.0	60.0	76.7	46.7	76.7
9	Documentation	56.7	86.7	36.7	80.0	53.3	73.3
Average		60.4	82.2	54.8	79.3	51.5	77.4

achieved a κ of 0.54 across runs, indicating a moderate level of consistency and alignment.

Both *gpt-4o* and *gpt-5-mini* exhibit higher agreement across runs (0.76 for *gpt-4o* and 0.74 for *gpt-5-mini*), indicating greater stability in the grading process.

4.3 Agreement across Indicators

In this section, we address the research question RQ3: *Does the agreement between human and LLM grading vary across different rubric indicators?* Table 3 shows the agreement between the grades assigned by the teacher and the grades generated by each LLM for each indicator. The agreement is calculated as the percentage of cases where the LLM grade matches the teacher grade for that indicator. LLMs and teacher exactly agreed in 27.4% of cases for *gpt-4o*, 25.9% for *gpt-5-mini*, and 20.0% for *qwen-coder:32b*. We also measured the partial agreement, which consid-

ers cases where the LLM converged on a level at least 3 times out of 5 runs, achieving an agreement of 46.3% for *gpt-4o*, 43.7% for *gpt-5-mini*, and 38.9% for *qwen-coder:32b*. Finally, we measured the tolerance agreement, which considers cases where the LLM grade is within one level of the teacher grade, achieving an agreement of 85.2% for *gpt-4o*, 83.7% for *gpt-5-mini*, and 74.8% for *qwen-coder:32b*. These results suggest that the agreement between LLM and teacher grading varies generally of one level out of six, but considering that a category is composed from several indicators, the overall agreement and the difference between the final grades assigned by the LLM and the teacher is much lower, as shown in Table 1. However, indicators like *Maintainability*, *Naming*, *Modularity* and *Formatting* showed a higher tolerance agreement between LLMs and teacher, with tolerance agreement ratios across all three models exceeding 80%.

Figure 1 also reports the agreement between each

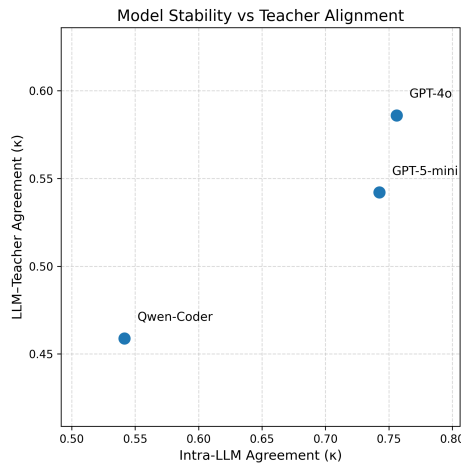


Figure 1: Comparison of intra-LLM stability and LLM–teacher agreement measured using Cohen’s κ . Each point represents a model (gpt-4o, gpt-5-mini, qwen-coder:32b). The x-axis reports the agreement across five runs of the same LLM (stability), while the y-axis reports the agreement between the LLM and the teacher grades (alignment).

LLM and the teacher grades, shown on the y-axis and measured using Cohen’s κ . The results indicate a moderate level of agreement with the teacher across all models. In particular, *gpt-4o* achieves the highest agreement ($\kappa = 0.59$), followed by *gpt-5-mini* (0.54) and *qwen-coder:32b* (0.46). These results suggest that, although the models exhibit internal consistency across runs, their grading decisions do not fully align with the teacher annotations.

5 DISCUSSION

The results presented in the Section 4 highlight several key insights regarding the use of LLMs for automatic grading of programming assignments. Using a rubric-based approach, we observed that the difference between LLM-generated grades and teacher grades varies across indicators, with *gpt-4o* showing the lowest overall difference with the human teacher (0.16 out of 10), followed by *gpt-5-mini* (−0.5 out of 10) and *qwen-coder:32b* (−0.64 out of 10). These results suggest that a rubric-based approach can help LLMs to generate more aligned grades with human evaluators. In Piscitelli et al. (2025), the authors evaluated code assignments without using a rubric approach, showing a higher difference between LLM and teacher grades. However, the results also show that, for some indicators, LLMs have difficulty aligning with teacher-assigned grades. In particular, *Specification Adherence* and *Design Autonomy* exhibit

larger discrepancies across all three models, suggesting that LLMs may struggle to fully interpret and apply assignment requirements. These indicators require a deeper understanding of the task context and implicit design expectations, which may not be fully captured by the input representation provided to the models.

In contrast, indicators such as *Modularity*, *Formatting*, and *Documentation* show smaller differences with respect to teacher evaluations. These aspects are more directly observable from the code and less dependent on contextual interpretation, making them more amenable to LLM-based assessment. This suggests that LLMs may be particularly effective in evaluating well-defined and structurally grounded dimensions of code quality. Further analysis is needed to understand the reasons behind the observed differences, in order to explore the potential improvements in LLM-based grading systems: for example, it will be possible to analyse different indicators with different LLMs.

Results also analysed the stability of the grading process across multiple runs. As described in Section 3.3, we ran each LLM five times with the same configuration and dataset and, at the end, we analysed the agreement across runs, computing the Cohen’s κ to measure the consistency of the grading process. These results showed that, despite the total agreement across runs was around 50% for all three models, at least 4 out of 5 runs converged on the same grade in more than 77% of cases (with *gpt-4o* achieving the highest partial agreement of 82%). Moreover, we also analysed the majority agreement, which considers cases where the LLM converged on the same level in at least 3 out of 5 runs, achieving the agreement of 100% for almost the indicators for *gpt-4o* and *gpt-5-mini*, and almost all for *qwen-coder:32b*. These results suggest that, although there is some variability in the grading process across runs, the majority of runs converge on the same grade, indicating a reasonable level of stability in the LLM-based grading process. However, the Cohen’s κ , reported in Figure 1, suggested that the agreement across runs is moderate for *qwen-coder:32b* (0.54) and substantial for *gpt-4o* (0.76) and *gpt-5-mini* (0.74), indicating that the grading process is relatively stable across runs, but there is still some variability that may affect the reliability of the grades generated by LLMs.

In the last analysis, we also measured the agreement between LLM and teacher grades across different indicators using Cohen’s κ . On the y-axis of Figure 1, the Cohen’s κ between LLM and teacher grades shows moderate agreement for all three models, with *gpt-4o* achieving the highest agreement (0.59),

Table 3: Agreement between LLM and teacher grades by indicator. EA = Exact Agreement, percentage of exact matches, MA = majority agreement (at least 3/5 runs match teacher), TA = tolerance agreement (within one level of difference between LLM and teacher grades).

ID	Indicator	GPT-4o			GPT-5-mini			Qwen-Coder		
		EA(%)	MA(%)	TA(%)	EA(%)	MA(%)	TA(%)	EA(%)	MA(%)	TA(%)
1	Functional Completeness	33.3	43.3	76.7	30.0	46.7	76.7	33.3	50.0	83.3
2	Robustness	30.0	53.3	80.0	30.0	43.3	83.3	16.7	36.7	73.3
3	Design	20.0	30.0	83.3	13.3	23.3	70.0	23.3	33.3	73.3
4	Autonomy									
5	Specification Adherence	16.7	33.3	60.0	10.0	20.0	76.7	20.0	43.3	53.3
6	Modularity	46.7	63.3	100.0	40.0	60.0	100.0	6.7	30.0	100.0
7	Maintainability	26.7	43.3	100.0	30.0	53.3	96.7	6.7	20.0	60.0
8	Naming	26.7	56.7	100.0	26.7	56.7	93.3	30.0	60.0	100.0
9	Formatting	33.3	63.3	100.0	46.7	63.3	96.7	33.3	60.0	90.0
	Documentation	13.3	30.0	66.7	6.7	26.7	60.0	10.0	16.7	40.0
Average		27.4	46.3	85.2	25.9	43.7	83.7	20.0	38.9	74.8

followed by *gpt-5-mini* (0.54) and *qwen-coder:32b* (0.46). These results suggest that, although the models exhibit internal consistency across runs, their grading decisions do not fully align with teacher annotations, indicating room for improvement in LLM-based grading systems.

Although the exact agreement between LLM and teacher grades was relatively low, when considering a tolerance of one level, the situation improves significantly: in more than 75% of cases, the LLM assigned a level at most one level different from the teacher grade, with *gpt-4o* achieving the highest tolerance agreement (85.2%), followed by *gpt-5-mini* (83.7%) and *qwen-coder:32b* (74.8%). We decided to consider a tolerance of one level on the ordinal scale because also human graders may have some variability in their grading decisions, and a one-level difference on a 6-level scale may be considered acceptable in many educational contexts, Messer et al. (2025). These results suggest that, although there is some variability in the grading decisions of LLMs, they are generally able to capture the overall quality of submissions in a way that is reasonably aligned with human grading, especially when considering a tolerance of one level on the ordinal scale.

Although a one-level difference on a 6-level scale may appear substantial, it is important to consider that each rubric category comprises several indicators, and the final grade is a weighted average. Consequently, the overall difference becomes smaller when considering category-level grades, as shown in Table 1.

5.1 Limitations

The findings of this study suggested that LLMs can be a valuable tool for automatic grading of programming assignments, especially when they use a rubric-based approach. However, there are still some limitations to consider; some indicators showed a higher difference between LLM and teacher grades, highlighting the limitations of using a single prompt to guide the grading process. In particular, indicators such as *Specification Adherence* and *Design Autonomy* require a deeper understanding of the task context and implicit design expectations, which may not be fully captured by the input representation provided to the models. This suggests that LLMs may struggle to fully interpret and apply assignment requirements, especially when they involve more complex and context-dependent judgments.

Moreover, in this study we used a relatively small dataset of 30 programming submissions, graded by a single teacher; this scenario represents a common situation in many educational contexts, where the grading process is mainly carried out by a single teacher, but it also limits the generalizability of the results. On the other hand, it is important to consider that the variability in human grading can be significant between different teachers, and the agreement between LLM and teacher grades may vary depending on the specific teacher's grading style and criteria. Future research should consider using a larger dataset of programming submissions, graded by multiple teachers, to better understand the variability in human grading and its impact on the agreement with LLM-generated grades, as well as to explore the potential of LLMs to capture the variability in human grading decisions.

In the last analysis, the experimental configuration differs slightly across models, particularly in terms of temperature settings (0.5 for GPT-4o and Qwen-Coer, 1.0 for GPT-5-mini), which may influence the variability of the generated outputs. These differences are due to practical constraints in model availability, and their impact is partially mitigated by the repeated-run analysis, which explicitly measures grading stability.

To address these limitations, further work should explore the use of a multi-agent approach, where each category of the rubric is evaluated by a different agent, specialised for that category. This approach may help to improve the alignment between LLM and teacher grades, especially for indicators that showed a higher difference in this study. Furthermore, a multi-agent framework could be employed to address these limitations, wherein each rubric category—or potentially each individual indicator—is evaluated by a specialised agent. Such a decomposition may enhance alignment between LLM-generated and teacher-assigned grades, particularly for indicators demonstrating substantial divergence in this investigation. Additionally, future research should consider using a larger dataset of programming submissions, graded by multiple teachers, to better understand the variability in human grading and LLM grading. Finally, it will be interesting to explore, in a user-study, the agreement with grading in a blind setting, where teachers will be asked to choose between two different grades (one generated by the LLM and one assigned by a human teacher).

6 CONCLUSIONS

In this study we investigated the use of LLMs for automatic grading of programming assignments, carried out by students during the course Programming and Data Structures, using a rubric-based approach. We analysed three different LLMs, comparing their generated grades with those assigned by a human teacher, on a dataset composed by thirty programming submissions. The experiment aimed to measure the agreement between LLM and teacher grades, the stability of the grading process across multiple runs, analysing these metrics across different indicators of the rubric. The results showed that:

- **RQ1.** Grade differences between LLMs and the teacher vary across indicators; however, *gpt-4o* shows the closest alignment, followed by *gpt-5-mini* and *qwen-coder:32b*. This suggests that rubric-based evaluation improves alignment, although some indicators remain challenging for LLMs.
- **RQ2.** Grading is reasonably stable across runs, with most outputs converging. Cohen's κ indicates moderate to substantial agreement, though some variability persists, potentially affecting reliability.
- **RQ3.** Agreement between LLMs and the teacher is moderate overall, with *gpt-4o* performing best. Allowing a one-level tolerance significantly increases agreement, indicating that LLMs generally capture overall submission quality despite minor grading differences.

Future work will attempt to analyse the limitations already presented in the previous section. We will explore the use of a multi-agentic approach, where each category (and maybe also each indicator) of the rubric is evaluated by a different agent, specialised for that category. We also want to explore the use of a larger dataset of programming submissions, graded by multiple teachers, to better understand the variability in human grading and LLM grading. Finally, it will be interesting to explore, in a user-study, the agreement with grading in a blind setting, where teachers will be asked to choose between two different grades (one generated by the LLM and one assigned by a human teacher). Finally, we also want to explore the use of LLMs for automatic generation of rubrics, either through the use of a DSL or through the use of LLMs themselves, to further simplify the grading process for teachers and to provide more transparent and explainable grading criteria for students.

ACKNOWLEDGMENT

We would like to acknowledge the valuable contribution of the student Marta Coiro. This work was partially supported by grants from the University of Salerno (grant number: 300392FRB25COSTA).

REFERENCES

- Akyash, M., Azar, K. Z., and Mardani Kamali, H. (2025). Stepgrade: Grading programming assignments with context-aware llms. In *2025 IEEE Integrated STEM Education Conference (ISEC)*, pages 1–6.
- Bloxham, S., den Outer, B., Hudson, J., and Price, M. (2016). Let's stop the pretence of consistent marking: exploring the multiple limitations of assessment criteria. *Assessment & Evaluation in Higher Education*, 41(3):466–481.
- Chapman, P. J., Rubio-González, C., and Thakur, A. V. (2024). Interleaving static analysis and llm prompting. In *Proceedings of the 13th ACM SIGPLAN In-*

- ternational Workshop on the State Of the Art in Program Analysis, SOAP 2024, page 9–17, New York, NY, USA. Association for Computing Machinery.
- Cisneros-González, J., Gordo-Herrera, N., Barcia-Santos, I., and Sánchez-Soriano, J. (2025). Jorgpt: Instructor-aided grading of programming assignments with large language models (llms). *Future Internet*, 17(6).
- Douce, C., Livingstone, D., and Orwell, J. (2005). Automatic test-based assessment of programming: A review. *J. Educ. Resour. Comput.*, 5(3):4–es.
- Edwards, S. H. (2003). Improving student performance by evaluating how well students test their own programs. *J. Educ. Resour. Comput.*, 3(3):1–es.
- Falchikov, N. and Goldfinch, J. (2000). Student peer assessment in higher education: A meta-analysis comparing peer and teacher marks. *Review of Educational Research*, 70(3):287–322.
- Grandel, S., Schmidt, D. C., and Leach, K. (2024). Applying large language models to enhance the assessment of parallel functional programming assignments. In *Proceedings of the 1st International Workshop on Large Language Models for Code, LLM4Code '24*, page 102–110, New York, NY, USA. Association for Computing Machinery.
- Ihantola, P., Ahoniemi, T., Karavirta, V., and Seppälä, O. (2010). Review of recent systems for automatic assessment of programming assignments. In *Proceedings of the 10th Koli Calling International Conference on Computing Education Research*, Koli Calling '10, page 86–93, New York, NY, USA. Association for Computing Machinery.
- Jukiewicz, M. (2025). A systematic comparison of large language models for automated assignment assessment in programming education: Exploring the importance of architecture and vendor.
- Keuning, H., Jeuring, J., and Heeren, B. (2018). A systematic literature review of automated feedback generation for programming exercises. *ACM Trans. Comput. Educ.*, 19(1).
- Mahbub, T., Dghaym, D., Shankarnarayanan, A., Syed, T., Shapsough, S., and Zualkernan, I. (2024). Can gpt-4 aid in detecting ambiguities, inconsistencies, and incompleteness in requirements analysis? a comprehensive case study. *IEEE Access*, pages 1–1.
- Messer, M., Brown, N. C., Kölling, M., and Shi, M. (2025). How consistent are humans when grading programming assignments? *ACM Trans. Comput. Educ.*, 25(4).
- Mohamed, K., Yousef, M., Medhat, W., Mohamed, E. H., Khoriba, G., and Arafa, T. (2025). Hands-on analysis of using large language models for the auto evaluation of programming assignments. *Information Systems*, 128:102473.
- Pathak, A., Gandhi, R., Uttam, V., Ramamoorthy, A., Ghosh, P., Jindal, A. R., Verma, S., Mittal, A., Ased, A., Khatri, C., Nakka, Y., Devansh, Challa, J. S., and Kumar, D. (2025). Rubric is all you need: Improving llm-based code evaluation with question-specific rubrics. In *Proceedings of the 2025 ACM Conference on International Computing Education Research V.I*, ICER '25, page 181–195, New York, NY, USA. Association for Computing Machinery.
- Piech, C., Huang, J., Nguyen, A., Phulsuksombati, M., Sahami, M., and Guibas, L. (2015). Learning program embeddings to propagate feedback on student code. In Bach, F. and Blei, D., editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1093–1102, Lille, France. PMLR.
- Piscitelli, A., Costagliola, G., De Rosa, M., and Fuccella, V. (2024). Influence of large language models on programming assignments – a user study. In *Proceedings of the 2024 16th International Conference on Education Technology and Computers, ICETC '24*, page 33–38, New York, NY, USA. Association for Computing Machinery.
- Piscitelli, A., De Rosa, M., Fuccella, V., and Costagliola, G. (2025). Large language models for student code evaluation: Insights and accuracy. In *Proceedings of the 17th International Conference on Computer Supported Education - Volume 2: CSSEDU*, pages 534–544. INSTICC, SciTePress.
- Schneider, J., Schenk, B., and Niklaus, C. (2024). Towards llm-based autograding for short textual answers. In *Proceedings of the 16th International Conference on Computer Supported Education - Volume 1: CSSEDU*, pages 280–288. INSTICC, SciTePress.
- Singh, R., Gulwani, S., and Solar-Lezama, A. (2013). Automated feedback generation for introductory programming assignments. *SIGPLAN Not.*, 48(6):15–26.
- Tseng, E.-Q., Huang, P.-C., Hsu, C., Wu, P.-Y., Ku, C.-T., and Kang, Y. (2024). Codev: An automated grading framework leveraging large language models for consistent and constructive feedback. In *2024 IEEE International Conference on Big Data (BigData)*, pages 5442–5449.
- Xie, W., Niu, J., Xue, C. J., and Guan, N. (2026). Grade like a human: Rethinking automated assessment with large language models. In *Proceedings of the International Conference on Research in Adaptive and Convergent Systems, RACS '25*, New York, NY, USA. Association for Computing Machinery.
- Yousef, M., Mohamed, K., Medhat, W., Mohamed, E. H., Khoriba, G., and Arafa, T. (2025). Begrading: large language models for enhanced feedback in programming education. *Neural Computing and Applications*, 37:1027 – 1040.
- Zhai, X. (2023). Chatgpt for next generation science learning. *XRDS*, 29(3):42–46.

APPENDIX

System Prompt

The following system prompt is used to instruct the AI model to act as an evaluator for C programming assignments.

Act as an evaluator of C programming assignments applying the provided grading rubric.

INPUT

- submission: the complete C source code
- programOutput: the output produced by the program
- staticAnalysisErrors: compiler or linter warnings
- teacherTestOutput: the instructor compilation/runtime result
- isMakefileWorking: whether the Makefile works correctly

Use all available information to evaluate the submission according to the rubric.

GRADING SCALE

All indicators must be evaluated using the following scale:

0 = requirements not met

1 = Very poor

2 = Weak / major issues present

3 = Acceptable / basic requirements satisfied

4 = Good / solid implementation with minor issues

5 = Excellent / complete and well-designed solution

Use the scale consistently across all indicators.

GRADING RULE

{{GRADING.RULE}}

EVALUATION PROCESS

1. Inspect the code and program behaviour.
2. Identify strengths and weaknesses related to the indicator.
3. Select the most appropriate rubric level.
4. Provide a short justification in the comment field.

TASK DESCRIPTION

{{TASK-DETAIL}}

OUTPUT FORMAT

Return ONLY a JSON object exactly following this schema:

```

1  {"correctness": {"indicators": [
2      {
3          "id": 1,
4          "name": "Adherence to requirements",
5          "selected_level": "4", "score": 4,
6          "comment": "All basic tasks and most advanced tasks are correctly implemented."
7      },
8      {
9          "id": 2,
10         "name": "Technical soundness",
11         "selected_level": "5", "score": 5,
12         "comment": "Edge cases and errors are generally well-handled; good overall robustness."
13     },
14     {
15         "id": 3,
16         "name": "Originality of the solution",
17         "selected_level": "3", "score": 3,
18         "comment": "Solution is consistent with the requirements but lacks extensions or
19             particularly autonomous design choices."
20     },
21     {
22         "id": 4,
23         "name": "Compliance with submission instructions",
24         "selected_level": "3", "score": 3,
25         "comment": "Some formal inaccuracies, but the project is evaluable as a whole."
26     }
27 ]}, "maintainability": {"indicators": [
28     {
29         "id": 5,
30         "name": "Modularity",
31         "selected_level": "4", "score": 4,
32         "comment": "Good separation of concerns with rare overlaps."
33     },
34     {
35         "id": 6,
36         "name": "Extensibility",
37         "selected_level": "4", "score": 4,
38         "comment": "The code can be modified with limited effort."
39     }
40 ]}, "readability": {"indicators": [
41     {
42         "id": 7,
43         "name": "Naming clarity",
44         "selected_level": "3", "score": 3,
45         "comment": "Names are understandable but sometimes generic."
46     },
47     {
48         "id": 8,
49         "name": "Formatting and style",
50         "selected_level": "4", "score": 4,
51         "comment": "Clear formatting with minor imperfections."
52     }
53 ]}, "documentation": {"indicators": [
54     {
55         "id": 9,
56         "name": "Comment quality",
57         "selected_level": "2", "score": 2,
58         "comment": "Limited comments; descriptions of functions are missing."
59     }
60 ]}

```